

Day 2 — DSA (C++)

Topics Covered

1. Binary Search
2. Search Insert Position
3. Time and Space Complexity
4. Pointers in C++
5. References in C++
6. Pointer to Structure — Introduction

1. Binary Search

Problem: Search for a target element in a sorted array and return its index. If the target does not exist, return `-1`.

Approach:

- Initialize `low = 0` and `high = n - 1`.
- Calculate the middle index.
- If the middle element equals the target, return its index.
- If the target is smaller, search the left half.
- Otherwise, search the right half.
- Repeat until the target is found or the search space becomes empty.

Time Complexity: $O(\log n)$

Auxiliary Space Complexity: $O(1)$ for the iterative approach.

2. Search Insert Position

Problem: Given a sorted array and a target value, return the target's index if it exists. Otherwise, return the index where it should be inserted to maintain sorted order.

Approach:

- Apply Binary Search.
- If the target is found, return its index.
- If the target is smaller than the middle element, move **high** to **mid - 1**.
- Otherwise, move **low** to **mid + 1**.
- When the loop ends, **low** represents the correct insertion position.

Time Complexity: $O(\log n)$

Auxiliary Space Complexity: $O(1)$.

3. Pointers in C++

A pointer is a variable that stores the memory address of another variable.

Example

```
#include <iostream>
using namespace std;
```

```
int main() {
    int a = 10;
    int *p = &a;

    cout << a << endl;
    cout << *p << endl;

    return 0;
}
```

Output:

```
10
10
```

Important Concepts:

- **&a** gives the address of **a**.

- `int *p` declares a pointer to an integer.
- `*p` accesses the value stored at the address held by `p`.

4. References in C++

A reference is another name, or alias, for an existing variable.

Example

```
#include <iostream>
using namespace std;
```

```
int main() {
    int a = 10;
    int &ref = a;

    ref = 50;

    cout << a << endl;
    cout << ref << endl;

    return 0;
}
```

Output:

```
50
50
```

Key Learning: Modifying the reference modifies the original variable.

5. Pointer vs Reference

Feature	Pointer	Reference
Stores an address	Yes	No, it acts as an alias

Declaration	<code>int *p =</code>	<code>int &r = a;</code> <code>&a;</code>
Can be reassigned to refer to another variable	Yes	No
Can represent no object	Yes, using <code>nullptr</code>	Not as a valid ordinary reference
Access value	<code>*p</code>	Use the reference name

6. Pointer to Structure

A structure groups related data members under one type. A pointer can store the address of a structure variable.

Example

```
#include <iostream>
using namespace std;

struct Student {
    int roll;
    float marks;
};

int main() {
    Student s = {1, 90.5};
    Student *p = &s;

    cout << p->roll << endl;
    cout << p->marks << endl;

    return 0;
}
```

Output:

1

90.5

The arrow operator `->` accesses a structure member through a pointer. For example, `p->roll` is equivalent to `(*p).roll`.

LeetCode Practice

1. #704 — Binary Search

The screenshot displays the LeetCode problem page for #704, Binary Search. The problem description states: "Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`. If `target` exists, then return its index. Otherwise, return `-1`. You must write an algorithm with $O(\log n)$ runtime complexity."

Example 1:
Input: `nums = [-1,0,3,5,9,12]`, `target = 9`
Output: `4`
Explanation: 9 exists in `nums` and its index is 4

Example 2:
Input: `nums = [-1,0,3,5,9,12]`, `target = 2`
Output: `-1`
Explanation: 2 does not exist in `nums` so return -1

Constraints:

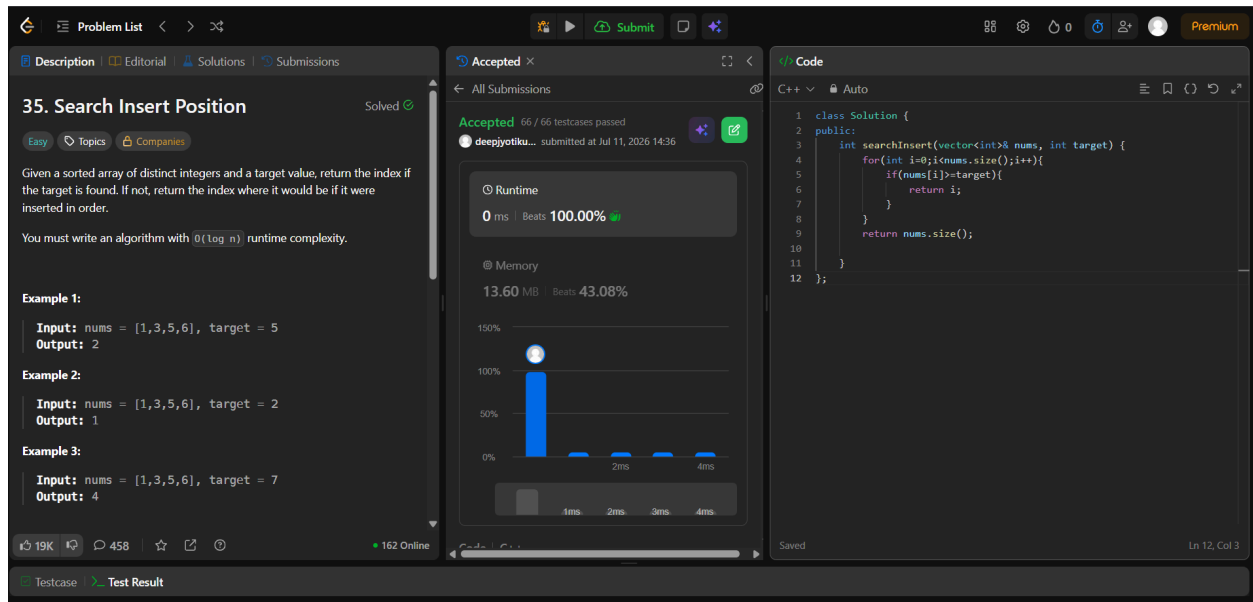
- $1 \leq \text{nums.length} \leq 10^4$
- $-10^4 < \text{nums}[i], \text{target} < 10^4$

The solution is implemented in C++:

```
1 class Solution {
2 public:
3     int search(vector<int>& nums, int target) {
4         for(int i=0; i<nums.size(); i++){
5             if(nums[i]==target){
6                 return i;
7             }
8         }
9         return -1;
10    }
11 }
```

The runtime is 0 ms, beating 100.00% of submissions. The memory usage is 31.27 MB, beating 81.56% of submissions. The solution is marked as "Accepted" and "Solved".

2. #35 — Search Insert Position



Key Learnings

- Understood the logic of Binary Search.
- Learned to solve problems on sorted arrays.
- Revised time and space complexity.
- Learned pointer declaration, address-of, and dereferencing.
- Understood references and how they modify original variables.
- Learned the basics of pointers to structures.

Revision Questions

1. Why does Binary Search require a sorted array?
2. Why is Binary Search $O(\log n)$?
3. What is the difference between `&` and `*`?
4. What is the difference between a pointer and a reference?
5. What does the `->` operator do?

1. Why does Binary Search require a sorted array?

Binary Search requires a sorted array because it compares the target with the middle element and eliminates half of the search space.

- If the target is smaller than the middle element, search the left half.
- If the target is greater, search the right half.

This works only when elements are arranged in sorted order.

Example: In the sorted array `[10, 20, 30, 40, 50]`, if the target is `40`, we can eliminate the left half after comparing it with the middle element.

2. Why is Binary Search $O(\log n)$?

Binary Search is $O(\log n)$ because it reduces the search space by approximately half in every iteration.

For example, if an array contains 16 elements, the search space reduces as follows:

`16 → 8 → 4 → 2 → 1`

The number of steps grows logarithmically with the number of elements.

- Time complexity: $O(\log n)$
- Auxiliary space complexity: $O(1)$ for iterative Binary Search.

3. What is the difference between `&` and `*` in C++?

The meaning depends on the context.

Ampersand (`&`):

- In `int &ref = a;`, it declares a reference.
- In `&a`, it obtains the memory address of `a`.

Asterisk (`*`):

- In `int *p;`, it declares a pointer.
- In `*p`, it accesses the value stored at the address held by the pointer.

Example:

```
int a = 10;  
int *p = &a;
```

```
cout << *p; // Output: 10
```

Here, `&a` obtains the address of `a`, and `*p` accesses its value.

4. What is the difference between a pointer and a reference?

A pointer is a variable that stores a memory address, whereas a reference is another name for an existing variable.

Pointer	Reference
Declared using <code>*</code>	Declared using <code>&</code>
Stores a memory address	Acts as an alias for a variable
Can be reassigned to point to another object	Cannot be reseated to refer to another object
Can be <code>nullptr</code>	A valid ordinary reference must refer to an object
Uses <code>*p</code> to access the pointed-to value	Uses the reference name directly

Example:

```
int a = 10;
```

```
int *p = &a;
```

```
int &ref = a;
```

```
*p = 20;
```

```
ref = 30;
```

```
cout << a; // Output: 30
```

Both can be used to modify the original variable, but their syntax and behavior differ.

5. What does the **->** operator do?

The arrow operator (**->**) accesses a member of a structure or class through a pointer.

Example:

```
struct Student {  
    int marks;  
};
```

```
Student s = {90};
```

```
Student *p = &s;
```

```
cout << p->marks;
```

Output: 90

The expression `p->marks` is equivalent to `(*p).marks`.

The arrow operator is useful when working with pointers to structures and objects.